

AlphaQuark — Architecture Consultation Brief

Independent review of trade & model-portfolio execution · prepared for scoping and quotation

Date: 19 September 2026 · **Status:** pre-engagement scoping document · **Confidentiality:** shared for the purpose of assessing and quoting an engagement only; please do not redistribute. A deeper technical evidence pack and read-only source access follow an NDA.

1. Why we are writing to you

AlphaQuark is a platform on which SEBI-registered advisers publish investment advice and their retail subscribers execute that advice in their own broker accounts, one tap at a time. The advice-to-execution loop is the product's core, and it is also where a design mistake costs a customer real money. We have built and hardened this loop in-house over several years and several production incidents; we now want an **independent, senior architect** to review the execution design end-to-end, tell us what is sound, what is not, and what the target design and migration path should be.

This document gives you the company context, the team and technology setup, a plain-English description of the system under review, and the specific areas where we want guidance — enough to assess fit and propose an engagement structure and price (hourly, fixed, or outcome-based; we are open to any of these). Section 8 lists what we would like back from you.

2. About AlphaQuark

What the product does

AlphaQuark is a B2B2C SaaS platform for **SEBI-registered Research Analysts (RAs) and Investment Advisers (IAs)** in India. An adviser runs their practice on AlphaQuark under their own brand: subscription plans and payments, KYC and agreement e-signing, publishing advice, and delivering it to subscribers over web, mobile app, WhatsApp and Telegram. Subscribers receive the advice and can execute it directly into their own brokerage account. AlphaQuark is **non-custodial**: we never hold client money or securities; every order is placed in the customer's own broker account under the customer's own authorization.

Advice comes in four shapes, all of which are executable:

- **Single-stock recommendations** — buy/sell with entry, stop and target; optional GTT/OCO on supported brokers.
- **Multi-leg derivatives baskets** — F&O strategies (spreads, straddles, condors) executed as a set of legs with independent broker outcomes and later closed as a set.
- **Model portfolios** — the adviser publishes target weights; each subscriber sets a funding amount; the platform computes the exact buy/sell deltas for that customer's account and executes them; subsequent rebalances and repairs keep the customer's holdings aligned with the model. *This is the most complex and highest-value flow.*

- **Mutual funds** — advisory and (behind gates) distribution rails; *out of scope for this review*.

Business model and footprint

- Advisers pay a platform fee; most run as **white-label** deployments (own domain, branding, and in several cases their own iOS/Android app), the rest on a shared multi-tenant build. Roughly two dozen adviser tenants are live today, each with an isolated database.
- **14 brokers** are integrated for direct API execution (Zerodha, Angel One, Upstox, ICICI Direct, Kotak, Dhan, Fyers, IIFL, AliceBlue, Motilal Oswal, HDFC Securities, Groww, Axis, plus an in-house test broker), and a separate **Zerodha Kite Publisher** rail where the customer confirms a pre-built basket inside Kite.
- Regulatory surface: SEBI RA/IA regulations — advice logging, mandatory NSE/CARE PaRRVA verification of every published advice, CVL-KRA KYC, MITC e-sign, invoice/GST, and five-year record retention. Execution correctness is therefore both a capital-risk and a compliance matter.

Why this review, and why now. Execution is the differentiator that sells the product, and it is where our worst incidents have occurred: a wrong quantity on a rebalance, a duplicate order after a lost response, a model selling another model's shares in the same broker account. We have fixed each class as it surfaced and we now have a much stronger design (Section 5), but it has never been assessed by someone outside the team. We want a credible, evidence-based verdict before we scale the number of advisers and brokers running model portfolios live.

3. Current team and technology setup

Team

AlphaQuark is a five-person company. **Pratik** leads product and technology and is hands-on across all three services; **Ankita** leads the business — adviser relationships, commercial and regulatory. Both will be in the room for this engagement.

- **Engineering** — Pratik plus two developers, full-stack across the React web app, Node.js API and Python services. This three-person group is your day-to-day counterpart and the team that will implement your recommendations.
- **Business** — Ankita: adviser relationships, commercial and regulatory; she owns the customer-impact side of any finding.
- **Sales and operations** — one person: adviser onboarding and customer support.

Engineering works with heavy use of AI coding agents under an unusually strict documentation discipline: every behaviour-changing commit must update the corresponding architecture document in the same commit, and every production incident has a written record. There is no dedicated SRE/QA function; the review's recommendations need to be executable by a team of this size.

Technology

Layer	What runs there
Customer / adviser web	React single-page app (one shared codebase, white-labelled per adviser); also drives Zerodha Publisher in the browser.
Mobile	React Native apps (a flagship app plus per-adviser forks) with OTA updates; host native Publisher and broker login flows.
Node.js API	Authentication (Firebase), entitlement and tenant scoping, authoritative advice reads, the broker-connection boundary, Publisher record-back, admin and support APIs, payment gateways.
Python / Flask + Celery	Rebalance calculation, broker adapters and order placement for all 14 brokers, durable jobs, reconciliation, corporate actions, scheduled crons, alerts.
Data	MongoDB (one database per adviser plus a shared platform database), Redis (Celery, leases, quotas, caches), PostgreSQL (SEBI recommendation log), object storage for documents.
Infra	Two Linux VPS hosts (frontend/nginx and backend services), systemd-managed services, Cloudflare in front, single region. Deploys are scripted and audited; no Kubernetes.

~1.1M

lines of first-party code across the three core repositories (web ≈460k, Node ≈240k, Python ≈380k)

14 + 1

broker API integrations plus the Zerodha Kite Publisher rail

~24

adviser tenants, each an isolated database and (mostly) white-label deployment

27

maintained architecture documents, plus dated incident records and diagram sources

4. The system under review, in plain English

The review scope is the path from a customer saying "yes" to an advice, to the platform being able to say with evidence what happened in the customer's broker account and what the customer now holds under each model.

4.1 Model-portfolio lifecycle

1. **Adviser publishes a model version** — constituents, target weights, cash allocation.
2. **Customer's funding instruction is read** — first investment, top-up, "set total to X", or continue with existing model capital.
3. **Customer connects a broker** — credentials/OAuth per broker; session is verified, not assumed from a stored "connected" flag.
4. **Calculate** — the server computes a per-customer, per-broker delta from model weights, attributed holdings and cash, live prices, whole-share constraints and settlement rules, and **freezes** it as an immutable plan the customer reviews and consents to.
5. **Execute** on one of two rails:
 - **Direct broker API** — the server places SELLS first, then the frozen BUYs, through the broker adapter; broker acceptance/rejection is the buying-power authority.

- **Zerodha Publisher** — the browser/app opens a pre-built basket inside Kite; the customer confirms there. The server prepares an intent before the popup, records what came back, and reconciles against the correct broker account's order book. The browser is a participant, not the coordinator.
6. **Record and allocate** — order IDs, tags and cumulative fills are matched to the frozen legs and written to the customer's per-model holdings book (a versioned, transactional record), with separate projections for execution status and dashboard state.
 7. **Rebalance / Repair later** — a new model version triggers a fresh Calculate; an incomplete execution is repaired from the exact frozen remainder, never by re-sending the original order set.

4.2 Single-stock and basket execution

Baskets are collections of legs with independent broker outcomes. Before placement the server reloads the authoritative advice, freezes its version, validates expiry/product/lot units and computes each leg's exact remaining quantity from recorded fills. Basket status is derived from every leg. Closure is sized to the customer's verified net open quantity — not the advised quantity, because customers fill differently. A retry after an ambiguous outcome becomes cancel-plus-reconcile, never a replacement order in the same attempt.

4.3 Execution rails at a glance

Rail	Who accepts	Who places	Who records the result	Present posture
Direct API — single & basket	Durable job in the Python service (idempotent)	Broker-specific worker	Result writer + reconciler	Live in production
Direct API — model portfolio	Frozen plan + per-leg claims	Synchronous broker adapter	Transactional holdings-book writer with legacy mirrors	Live; account-by-account guarded rollout
Zerodha Publisher — model & basket	Prepared intent + exact batch activation	Customer gesture inside Kite	Idempotent record-back + correct-account order-book reconciler	Live; unresolved outcomes are held, not guessed
GTT / OCO	Separate broker-specific path with polling			Limited; not covered by general approval

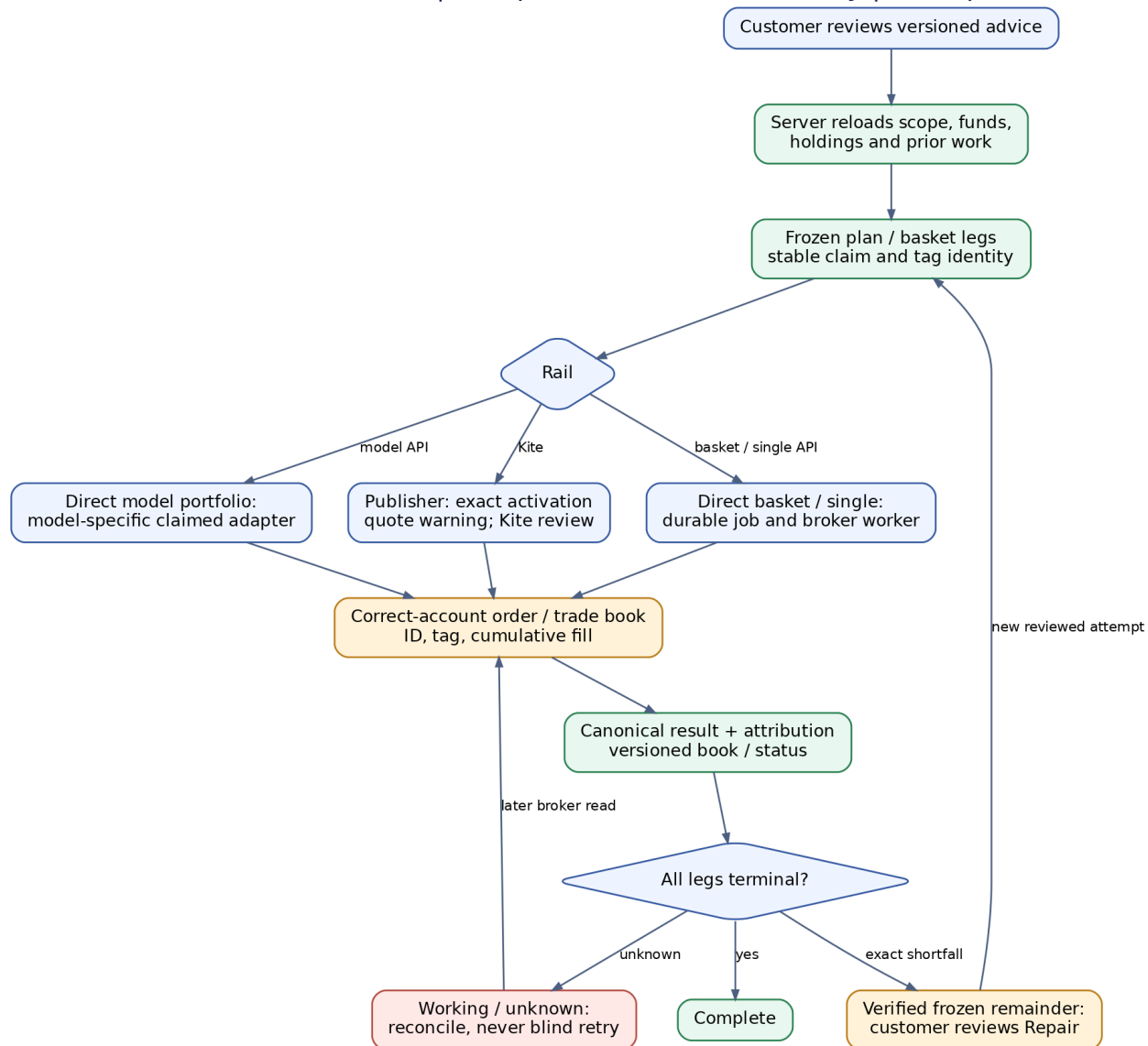


Figure 1 — Execution authority across the three rails: frozen intent → rail-specific placement → correct-account broker evidence → canonical result → complete / reconcile / repair.

4.4 The hard problems

- **One broker account, many owners.** A customer's single demat/trading account holds their personal shares plus positions from several model portfolios (possibly from more than one adviser). Live holdings alone cannot say which model owns which share; ownership must come from execution evidence. Today, ambiguous cases fail closed rather than auto-allocating by symbol.
- **Lost responses and partial truth.** A broker may accept an order and then time out; a callback may arrive twice or out of order; a fill may be partial. The design distinguishes "definitely failed" from "unknown", and "unknown" is reconciliation work, not a retry.
- **The browser as a participant.** On the Publisher rail the customer's tab or phone is part of the transaction. Tab close, app death, double clicks, multiple devices and CDSL authorization redirects must all converge on the same server-side attempt.
- **Financial truth vs. projections.** The canonical per-model holdings book is served through a versioned reader; older compatibility snapshots and status mirrors still exist. An

append-only event ledger, replay projector and invariant checker exist as diagnostic witnesses but are not yet the universal authority. Whether and how to promote them is a central open question.

- **Settlement and margin** between SELL and BUY phases; corporate actions between ex-date and demat credit; broker quota and rate limits at scale.
- **Tenant isolation** across per-adviser databases, shared builds, support tooling and scheduled fleet-wide jobs.

5. What has already been built — so you can gauge maturity

This is not a greenfield design review. The following are in production or in shadow, each with code, tests and dated incident history behind it:

- Immutable, server-persisted plans; per-leg atomic claims before any broker call; durable dispatch markers.
- Idempotent acceptance for direct orders (same key + same payload → same result; different payload → conflict, never a new order).
- Publisher prepared-intent and exact-batch activation; typed attribution gaps for missing order IDs/tags; correct-account reconciliation.
- Transactional canonical holdings-book writer with a versioned shared reader; a documented precedence order between canonical book, checkpoints, execution evidence, live broker truth and legacy snapshots; a migration that backfilled and quarantined legacy books.
- Exact-remainder Repair distinct from recomputed-delta Calculate; intent-preserving drift correction within model capital.
- Basket partial-fill schema, cancel-before-retry, closure clamped to verified net position, unit accessors for lots vs. lot-size.
- Append-only execution event ledger, replay projector, order-book archive and invariant checker (diagnostic today).
- Scenario-driven test broker; per-broker QA suites; contract tests on the shared status vocabulary; twenty named invariants tracked against enforcement points.
- Alerting to an internal Telegram channel; audit collections for recoveries, anomalies and manual operations; a read-only support console.

We can share the full evidence pack (six review documents, a review workbook with decision tables and a failure/recovery matrix, thirteen versioned diagrams, and the underlying architecture documents) and read-only repository access once an NDA is in place.

6. Where we want your guidance

These are the questions we cannot credibly answer about our own work. They are grouped so you can scope each independently.

Area	The question we want answered
A. Execution-safety verdict, per rail	For each rail (direct basket, direct model portfolio, Zerodha Publisher, basket closure/retry, model repair, GTT): approve, approve with conditions, shadow-only, or do not approve — with the evidence you relied on and the conditions that would change the verdict. Where is the linearization point for acceptance, activation, dispatch, result and completion, and where there is none, what invariant plus compensating reconciliation stands in for it?
B. Financial truth and ledger authority	Should the append-only allocation/event ledger become the source of truth for per-model holdings and cash, with today's book as a projection? If so, what is the cutover contract and the gate for retiring compatibility writers and readers? What correction/audit model should replace in-place edits?
C. Identity, idempotency and multi-model allocation	Is the durable identity key (tenant · customer · broker · model · plan · attempt · leg) sufficient and consistently applied across calculate, execute, record-back, corporate actions, closure and manual repair? For the same symbol held by two models in one account: block, require account separation, or build attributed lots? Collision analysis on IDs across brokers, accounts and trading days.
D. Recovery and reconciliation	Are the failure paths (worker death before/after dispatch, broker timeout after acceptance, duplicate/late callbacks, token expiry, order-book unavailability, reconciler stall, Mongo/Redis loss) detected and handled without blind replay or false "not found"? What is an acceptable maximum time for an attempt to stay unresolved, and what evidence is sufficient to close it on the Publisher rail?
E. Broker adapter certification	Should broker/flow certification be a code-enforced gate rather than allowlist discipline? Per-broker order/product/exchange support, quota scopes, callback and order-book limitations, auth-expiry behaviour and what a controlled live certification must prove.
F. Derivatives sequencing and exposure	Leg sequencing policy for multi-leg baskets (advice order, sell-first, hedge-first, fill-gated) and its margin/capital consequences; visibility and control of naked exposure on partial closure.
G. Testing, observability and operations	Which invariant, concurrency, chaos and property tests are missing and worth writing first; what runtime monitors, SLOs, dashboards and alert ownership a five-person team can realistically run; what support may and may not assert without evidence.
H. Target architecture and migration sequence	A recommended end-state for the execution core and a sequenced migration that a three-engineer team can deliver incrementally without a big-bang rewrite, with rollout gates and rollback criteria at each step.

I. Security and tenant isolation	Customer/adviser/model/order boundaries, internal-token and secret handling, support-tool scopes, cron and fleet-scan paths, and shared-build tenant resolution.
---	--

7. Deliverables and how we would judge outcomes

We would rather receive a concise, linked set of documents than one long narrative. For an outcome-based proposal, the right-hand column is what "done" looks like to us.

Deliverable	Acceptance — what we would consider it complete
1. Architecture review brief	A decision (approve / with conditions / shadow only / do not approve) for each rail in Section 6-A, each traceable to code, schema, test or production evidence, with disagreements between documents and code recorded as findings.
2. Ranked risk register	Each risk with severity, likelihood, blast radius, evidence, customer/capital impact, owner, recommended action and acceptance criteria; the top three demonstrated as concrete scenarios.
3. Lifecycle, identity and financial-truth opinion	Answers to Sections 6-B, 6-C and 6-D that name the linearization point (or its absence) for every stage, and a decision on ledger promotion and multi-model allocation with a migration contract.
4. Broker adapter and failure/recovery matrices	Completed per-broker support/quota/limitation matrix and a completed failure → detection → automatic action → must-not-happen → alert owner → evidence retained matrix.
5. Target architecture and migration sequence	An end-state design plus an ordered set of steps, each with a rollout gate, a rollback path and an estimate of effort for a team of three engineers.
6. Test, monitoring and operations deltas	A prioritized list of tests to write (with what each proves), runtime monitors and alerts to add, and runbook changes — specific enough to become tickets.
7. Final walkthrough	A session with Pratik, Ankita and the developers covering decisions by rail, unresolved facts vs. accepted risks, immediate blocks, 30-day work and the structural roadmap; agreement on a re-review date.
8. (Optional) follow-up re-review	A shorter pass after remediation confirming that conditions were met and gates passed.

What we will provide

- Mutual NDA; read-only access to the three core repositories and the mobile apps; the full review pack and architecture documents.
- A staging environment with the scenario-driven test broker; redacted production evidence (metrics, reconciliation cases, incident records, audit collections) on request.
- Direct asynchronous access to the engineering team for questions, plus a weekly sync with Pratik and Ankita; developers available for code walkthroughs.

- An evidence-level convention we already use (design intent → code → automated test → controlled broker evidence → production record) so that findings can be classified consistently.

Out of scope

Portfolio-selection alpha, marketing pages, visual design, mutual-fund rails, and non-order payment features — unless needed to validate an execution boundary.

8. What we would like back from you

1. Your **proposed approach** to the review and the order in which you would take the areas in Section 6.
2. The **engagement structure and pricing** you prefer — hourly, fixed fee, or outcome-based against the deliverables in Section 7 — and any assumptions behind it (e.g. how much of Section 6 you would cover, and what would be an add-on).
3. Your **availability** and an indicative elapsed timeline.
4. A note on **relevant prior work** — ideally the hardest retry, reconciliation or financial-truth problem you have reviewed and how you validated the final design — and one redacted sample of a technical review or design document.
5. Anything in this brief you need clarified before quoting; we are happy to do a short call first.

Confidentiality. The concepts, workflows and design decisions described here are AlphaQuark intellectual property, shared solely to let you assess and quote this engagement. Please do not share this document outside your own practice. Source code, schemas, production data and the detailed evidence pack are provided only under NDA.

Contact: Pratik and Ankita · pratik@alphaquark.in